FULL STACK AI CHEAT SHEET + 100 INTERVIEW QUESTIONS

Complete Guide

1. Frontend

React | Next.js | Tailwind CSS

HTML | CSS | JavaScript | TypeScript

2. AI / LLM

OpenAI | Hugging Face | Llama

LLMs | RAG | Prompt Engineering
Fine-tuning | AI Agents

3. Backend

Node.js | Python | FastAPI

Express.js | REST APIs | WebSockets
Microservices | Serverless

4. Database

PostgreSQL | MongoDB | Pinecone

SQL | NoSQL | Vector DB
Embeddings | Similarity Search

5. Tools & Frameworks

LangChain | LangGraph

Agents | Chains | Workflows
Tools | Memory | Orchestration

6. Cloud & Deployment

aws | GCP | Azure | Vercel | Docker | Kubernetes

CI/CD | DevOps | Production
Scalability | Monitoring

7. AI Applications

Chatbot | RAG | Image Gen

Document Q&A | AI Agents
AI SaaS Products | Automation

8. Security & Auth

JWT | OAuth | RBAC

Authentication | Authorization
API Security | Best Practices

9. Monitoring & DevOps

Prometheus | Grafana | ELK Stack

Logs | Metrics | Tracing
Scaling | Uptime | Alerts

Concepts | Code | Architecture | Best Practices
Real World Use Cases | Deployment

@Codedsoul_05



1. Full Stack AI Developer Introduction + Roadmap

What is a Full Stack AI Developer ?

- A Full Stack AI Developer is a developer who builds complete AI powered applications from frontend to backend.
- It combines full stack web development skills with AI and machine learning capabilities.
- You can build, deploy and maintain AI applications end to end.
- It is one of the most in demand and high paying roles in the tech industry.

Why is it Important ?

- AI is transforming every industry (healthcare, finance, education, e-commerce).
- Companies need developers who can build complete AI products, not just AI models.
- Combines high demand skills (web + AI).
- Offers better career opportunities and higher salary.
- You can work on real world products and startups.

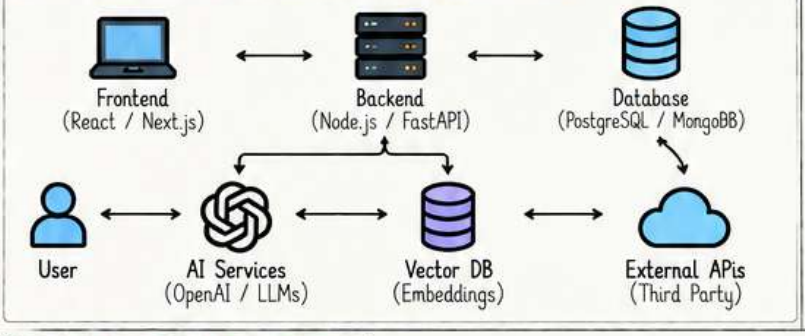
Key Skills Required

1. **Frontend Development** (HTML, CSS, JavaScript, React, Next.js)
2. **Backend Development** (Node.js, Express, Python, FastAPI)
3. **Databases** (SQL, PostgreSQL, MongoDB)
4. **AI and Machine Learning** (ML, LLMs, Prompt Engineering)
5. **API Integration** (REST APIs, OpenAI APIs, LangChain)
6. **Deployment** (Docker, Cloud, Vercel, AWS)
7. **Tools and DevOps** (Git, GitHub, CI CD)

Tech Stack

Frontend	HTML, CSS, JavaScript, React.js, Next.js, Tailwind CSS
Backend	Node.js, Express.js, Python, FastAPI
Database	PostgreSQL, MongoDB, Redis
AI / ML	OpenAI, LLMs, LangChain, LangChain, RAG, AI Agents
DevOps	Docker, Git, GitHub, AWS, Vercel

Full Stack AI Application Architecture



Types of Full Stack AI Applications

- **AI Chatbot** — Chat with documents, websites or custom data.
- **AI SaaS Product** — AI powered tools for businesses.
- **AI Automation Tool** — Automate tasks using AI agents.
- **AI Content Generator** — Generate text, images, videos and more.
- **AI Analytics Dashboard** — Analyze data using AI and visualize insights.
- **AI Agent Applications** — Multi agent systems for complex workflows.

Real World Examples

ChatGPT	Notion AI	GitHub Copilot	Perplexity AI	Midjourney
AI chatbot with web interface.	AI integrated productivity tool.	AI pair programmer for developers.	AI powered search engine.	AI image generation tool.

Career Opportunities

- Full Stack AI Developer
- AI Application Developer
- AI Engineer
- ML Engineer
- SDE with AI Focus
- Startup Founder

Step by Step Roadmap (15 Page Series)

1. Intro + Roadmap (This Page)
2. HTML CSS JS TypeScript
3. React.js + Next.js
4. Node.js + Express.js
5. Python + FastAPI
6. SQL + MongoDB
7. Auth + API Security
8. AI + ML + Fundamentals
9. LLMs + Prompt Engineering
10. OpenAI APIs + Integration
11. Embeddings + RAG
12. LangChain + AI Agents
13. Architecture + Best Practices
14. Deployment + Production
15. Cheat Sheet + 100 Questions

Learning Resources

Topic	Resource (Free)	Resource (Paid)
Web Development	MDN Web Docs	JavaScript Mastery
React / Next.js	Next.js Docs	Codevolution / Hitesh Choudhary
Backend (Node.js)	Node.js Docs	Hitesh Choudhary
Python / FastAPI	FastAPI Docs	ArjanCodes
AI / LLMs	OpenAI Docs	LangChain, DeepLearning.AI
Deployment	Vercel / Docker Docs	Kubernetes, AWS Tutorials

Tips for Success

- Build real projects while learning.
- Focus on hands on practice, not just theory.
- Learn to integrate AI into full stack applications.
- Use open source projects to gain experience.
- Keep updating with new AI tools and frameworks.
- Create a portfolio and showcase your projects.
- Follow tech communities and stay consistent.

@codedsoul_05

2. HTML + CSS + JavaScript + TypeScript

@codedsoul_05

What is HTML ?

- HTML (HyperText Markup Language) is the standard language for creating web pages.
- It provides the structure and content of a webpage.
- Uses tags to define elements like headings, paragraphs, images, links, forms etc.
- Example: <h1>, <p>, , <a>

What is CSS ?

- CSS (Cascading Style Sheets) is used to style and design web pages.
- It controls the layout, colors, fonts, spacing and responsive design.
- It separates the design from the content (HTML).
- Example: color, font-size, margin, display, flex, grid

What is JavaScript ?

- JavaScript is a programming language that makes web pages interactive.
- It can manipulate HTML and CSS, handle events, work with APIs and create dynamic content.
- It runs in the browser and also on the server (Node.js).
- Example: variables, functions, events, DOM, fetch API

What is TypeScript ?

- TypeScript is a superset of JavaScript developed by Microsoft.
- It adds static typing to JavaScript.
- Helps catch errors early and makes code more maintainable.
- Compiles to JavaScript and runs in the browser.
- Example: let name: string = "Nikhil";

HTML Basic Structure

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My Website</title>
</head>
<body>
  <h1>Hello World</h1>
  <p>This is a paragraph.</p>
</body>
</html>
```

CSS Example

```css
h1 {
  color: #3498db;
  font-size: 24px;
  margin: 10px 0;
}

box {
  background: #f1f1f1;
  padding: 20px;
  border-radius: 8px;
}
```

JavaScript Example

```javascript
// Variables
let name = "Nikhil";

// Function
function greet() {
  console.log("Hello " + name);
}
greet();

// DOM Manipulation
document.getElementById("demo").
.innerText = "JavaScript is fun!";
```

TypeScript Example

```typescript
let username: string = "Nikhil";
let age: number = 24;
let isActive: boolean = true;

function greetUser(name: string);
 string {
  return `Hello ${name}`;
}

console.log(greetUser(username));
```

Common HTML Tags

Tag	Purpose
<h1> to <h6>	Headings
<p>	Paragraph
<a>	Link
<img>	Image
	Container
	Inline container
, ,	Lists
<form>	Form
<input>	Input field
<button>	Button
<table>	Table
<header>, <footer>	Page layout

CSS Selectors

Selector	Example
Element	p { ... }
Class	.box { ... }
ID	#header { ... }
Universal	* { ... }
Attribute	input[type="text"]
Pseudo-class	a:hover { ... }
Pseudo-element	p::first-line { ... }
Group	h1, p { ... }

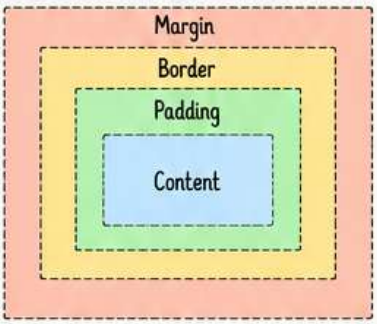
JavaScript Key Concepts

- Variables (let, const, var)
- Data types (string, number, boolean, array, object)
- Functions (normal, arrow, callback)
- DOM manipulation
- Events (click, input, submit)
- Arrays and array methods
- Objects and object methods
- Asynchronous JS (setTimeout, Promises, async/await)
- Fetch API for HTTP requests
- ES6+ features (destructuring, spread, modules)

TypeScript Key Concepts

- Type annotations
- Interfaces
- Types
- Enums
- Generics
- Type inference
- Union types (|)
- Intersection types (&)
- Type aliases
- Utility types (Partial, Pick, Omit)
- Type guards
- Working with existing JavaScript libraries

CSS Box Model



CSS Flexbox Example

```css
.container {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 10px;
}

.item {
  background: #3498db;
  color: white;
  padding: 10px 20px;
  border-radius: 6px;
}
```

CSS Grid Example

```css
.grid {
  display: grid;
  grid-template-columns:
    repeat(3, 1fr);
  gap: 10px;
}
.item {
  background: #2ecc71;
  padding: 20px;
  text-align: center;
  border-radius: 6px;
  color: white;
}
```

Responsive Design

```css
@media (max-width: 768px) {
  .container {
    flex-direction: column;
  }
  .grid {
    grid-template-columns:
      1fr;
  }
  h1 {
    font-size: 20px;
  }
}
```

JavaScript DOM Example

HTML

```html
<button id="btn">Click me</button>
<p id="text">Hello</p>
```

JavaScript

```javascript
document.getElementById("btn")
.addEventListener("click", () => {
  document.getElementById("text").
  .innerText = "Button Clicked!";
});
```

TypeScript Interface Example

```typescript
interface User {
  id: number;
  name: string;
  email: string;
  isActive: boolean;
}
```

```typescript
const user: User = {
  id: 1,
  name: "Nikhil",
  email: "nikhil@example.com",
  isActive: true
};
```

3. React.js + Next.js

@codedsoul_05

What is React.js ?

- React.js is a JavaScript library developed by Facebook for building user interfaces.
- It helps to build reusable UI components.
- Uses a virtual DOM for fast updates.
- It is component based, declarative and follows a unidirectional data flow.
- It is mainly used for single page applications (SPA).

Why React ?

- Component based architecture
- Reusable components
- Virtual DOM (better performance)
- Large community and ecosystem
- Easy to integrate with other libraries
- Used by top companies (Facebook, Instagram, Netflix, etc.)

What is Next.js ?

- Next.js is a React framework developed by Vercel.
- It provides server side rendering (SSR), static site generation (SSG) and API routes.
- It helps to build full stack web applications with React.
- It is SEO friendly and optimized for performance.

React Project Structure

```
my-react-app/
├── node_modules/
├── public/
│   ├── index.html
│   └── favicon.ico
├── src/
│   ├── components/
│   ├── App.js
│   └── index.js
├── package.json
└── README.md
```

Create React App

```
npx create-react-app my-app

cd my-app

npm start
```

Runs on http://localhost:3000

Functional Component

```
import React from 'react';

function Welcome() {
  return (
    <div>
      <h1>Hello React!</h1>
      <p>This is a functional component.</p>
    </div>
  );
}
export default Welcome;
```

JSX Example

```
const element = (
  <div className="card">
    <h2>JSX Example</h2>
    <p>JavaScript XML syntax to write HTML in React.</p>
  </div>
);
```

Props Example

```
function User({ name, age }) {
  return (
    <div>
      <h2>Name: {name}</h2>
      <p>Age: {age}</p>
    </div>
  );
}
<User name="Nikhil" age={24} />
```

Props help to pass data from parent to child component.

State in React

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
    </div>
  );
}
```

State is used to store and manage dynamic data in a component.

useEffect Hook

```
import React, { useEffect } from 'react';

function DataComponent() {
  useEffect(() => {
    console.log("Component mounted");

    return () => {
      console.log("Component unmounted");
    };
  }, []); // Empty dependency array

  return <h2>useEffect Example</h2>;
}
```

useEffect is used to handle side effects like API calls, event listeners, etc.

Common React Hooks

Hook	Purpose
useState	Manage state in component
useEffect	Handle side effects
useContext	Access global state
useReducer	State management (complex)
useRef	Access DOM / persist values
useMemo	Memoize expensive values
useCallback	Memoize functions
useLayoutEffect	Runs before paint
useImperativeHandle	Customize ref behaviour

React Router Example

```
npm install react-router-dom
```

```
import { BrowserRouter,
 Routes, Route, Link } from
'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
```

Next.js Project Structure (App Router)

```
my-next-app/
├── app/
│   ├── page.js (Home Page)
│   ├── about/
│   │   └── page.js
│   └── api/
│       └── hello/
│           └── route.js
├── components/
├── public/
├── next.config.js
└── package.json
```

Create Next.js App

```
npx create-next-app@latest my-next-app

cd my-next-app

npm run dev
```

Runs on http://localhost:3000

Next.js Page Example

```
// app/page.js
export default function Home() {
  return (
    <div>
      <h1>Welcome to Next.js</h1>
      <p>This is a home page.</p>
    </div>
  );
}
```

Key Features of Next.js

- Server Side Rendering (SSR)
- Static Site Generation (SSG)
- API Routes (Backend in Next.js)
- File based routing
- Built in image optimization
- SEO friendly
- Full stack development
- Middleware and advanced routing
- Optimized for production

Routing in Next.js (App Router)

```
// app/about/page.js
export default function About() {
  return <h1>About Page</h1>;
}
```

File based routing - just create a folder and add page.js file.

API Routes Example

```
// app/api/hello/route.js
export async function GET() {
  return Response.json({
    message: "Hello from Next.js API"
  });
```

API routes help to create backend endpoints in Next.js.

Data Fetching (SSR / SSG)

```
// Server Side Rendering
export default async function Home() {
  const res = await fetch("https://api.com/data");
  const data = await res.json();

  return (
    <div>
      <h1>{data.title}</h1>
    </div>
  );
```

Next.js vs React.js

Feature	React.js	Next.js
Type	Library	Framework
Rendering	Client Side (CSR)	SSR, SSG, CSR
Routing	React Router	Built in routing
API Routes	Not built in	Built in
SEO	Limited	Better (SSR/SSG)
Setup	create-react-app	create-next-app
Use Case	Frontend	Full stack

When to Use What ?

Use React.js when:

- You need a single page application
- Your project is mainly frontend
- You want more control over setup
- You are building dashboards or interactive web apps

Use Next.js when:

- You need SEO friendly websites
- You want server side rendering or static sites
- You need full stack features (API routes)
- You are building production ready applications

Real World Examples

Facebook (React) · Instagram (React) · Netflix (React) · Vercel (Next.js) · TikTok (Next.js)

Best Practices

- Use functional components and hooks
- Keep components small and reusable
- Follow proper folder structure
- Use TypeScript for better code quality
- Use environment variables
- Optimize images and performance
- Follow SEO best practices (Next.js)

4. Node.js + Express.js + REST APIs

@codedsoul_05

What is Node.js ?

- Node.js is a JavaScript runtime environment built on Chrome's V8 engine.
- It allows us to run JavaScript on the server side.
- It is event driven, non blocking and asynchronous.
- It is mainly used to build scalable and high performance backend applications.

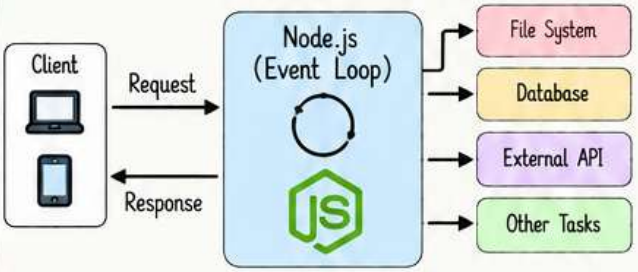
What is Express.js ?

- Express.js is a minimal and flexible web framework for Node.js.
- It helps to create web servers and APIs easily.
- It provides routing, middleware support, request and response handling and many built in features.
- It is the most popular Node.js framework.

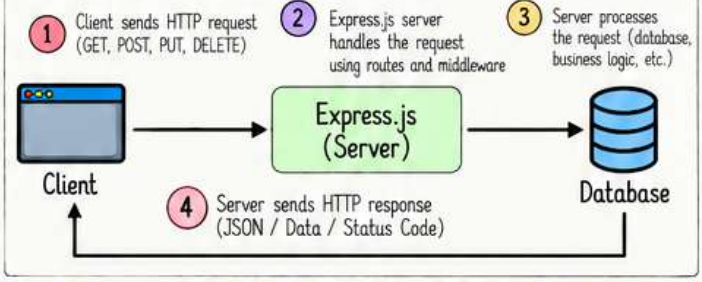
Why Node.js + Express.js ?

- JavaScript on both frontend and backend
- Fast and lightweight
- Huge community support
- Easy to learn and use
- Middleware and routing support
- Great for building REST APIs
- Scalable and suitable for real world projects
- Can work with databases easily (MongoDB, SQL)
- Widely used in industry (Netflix, LinkedIn, Uber etc.)

Node.js Architecture (Event Driven)



How Request Response Works ?



Setup Node.js Project

```
# 1. Create project folder
mkdir my-backend
cd my-backend

# 2. Initialize project
npm init -y

# 3. Install Express
npm install express

# 4. Create entry file
# index.js or app.js
```

Basic Express Server

```
// app.js
const express = require('express');
const app = express();
const PORT = 3000;

app.get('/', (req, res) => {
  res.send('Hello from Express.js!');
});

app.listen(PORT, () => {
  console.log(`Server running on
   http://localhost:${PORT}`);
});
```

Run Server

```
node app.js
```

Server will start at
http://localhost:3000

Middleware Example

```
const express = require('express');
const app = express();

// Built-in middleware to parse JSON
app.use(express.json());

// Custom middleware
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next(); // Pass to next middleware
});
```

HTTP Methods in REST API

Method	Purpose	Example
GET	Read data	/api/users
POST	Create new data	/api/users
PUT	Update entire data	/api/users/1
PATCH	Update partial data	/api/users/1
DELETE	Delete data	/api/users/1

CRUD Operations Example

```
const express = require('express');
const app = express();
app.use(express.json());

let users = [];

// Create (POST)
app.post('/api/users', (req, res) => {
  const user = req.body;
  users.push(user);
  res.status(201).json({ message: 'User created', user });
});
```

```
// Read (GET)
app.get('/api/users', (req, res) => {
  res.json(users);
});

// Update (PUT)
app.put('/api/users/:id', (req, res) => {
  const id = req.params.id;
  users[id] = req.body;
  res.json({ message: 'User updated', user: users[id] });
});

// Delete (DELETE)
app.delete('/api/users/:id', (req, res) => {
  const id = req.params.id;
  users.splice(id, 1);
  res.json({ message: 'User deleted' });
});
```

Route Parameters and Query Params

```
// Route Parameter
app.get('/api/users/:id', (req, res) => {
  const id = req.params.id;
  res.send('User ID: ${id}');
});

// Query Parameter
app.get('/api/search', (req, res) => {
  const { q, page } = req.query;
  res.send('Search: ${q}, Page: ${page}');
});
```

Error Handling

```
// Custom error handling middleware
app.use((err, req, res, next) => {
  console.errrer(err.stack);
  res.status(500).json({
    message: 'Something went wrong!',
    error: err.message
  });
});

// 404 Not Found
app.use((req, res) => {
  res.status(404).json({ message: 'Route not found' });
});
```

REST API Response Format

```
// Success Response
{
  "success": true,
  "message": "User fetched successfully",
  "data": { "id": 1, "name": "Nikhil" }
}
// Error Response
{
  "success": false,
  "message": "User not found",
  "error": "Invalid user ID"
}
```

Connecting to Database (MongoDB)

```
const mongoose = require('mongoose');

mongoose.connect('mongodb://localhost:27017/myapp')
  .then(() => console.log('MongoDB Connected'))
  .catch(err => console.log(err));
```

Environment Variables

1. Install dotenv
   `npm install dotenv`
2. Create .env file
   ```
   PORT=3000
   MONGO_URI=your_mongo_url
   ```
3. Use in code
   ```
   require('dotenv').config();
   const PORT = process.env.PORT;
   ```

Project Folder Structure

```
my-backend/
├── node_modules/
├── src/
│   ├── routes/
│   ├── controllers/
│   ├── models/
│   ├── middleware/
│   └── app.js
├── .env
└── package.json
```

Best Practices

- Use environment variables
- Follow MVC structure
- Use proper error handling
- Validate input data
- Use middleware (auth, logging, etc.)
- Keep routes modular
- Use async await
- Follow REST API standards
- Use tools like Postman for testing

5. Python + FastAPI for AI Applications

@codedsoul_05

Why Python for AI ?

- Simple and easy to learn
- Large community and ecosystem
- Rich libraries for AI and ML
- Widely used in industry and research
- Best for data processing, AI models, and automation

What is FastAPI ?

- Modern, fast and lightweight
- Built on top of Python and type hints
- Automatically generates API docs
- High performance (based on Starlette and Pydantic)
- Ideal for building AI and ML APIs

FastAPI

Why FastAPI for AI Applications ?

- Fast to build and easy to maintain
- Automatic Swagger documentation
- Supports async and high performance
- Easy integration with AI/ML models
- Handles large requests (files, images, text)
- Production ready and widely used in industry

Project Structure (AI API)

```
ai-fastapi-app/
├── app/
│   ├── main.py
│   ├── routes/
│   │   ├── __init__.py
│   │   └── ai_routes.py
│   ├── models/
│   │   ├── __init__.py
│   │   └── schemas.py
│   └── services/
│       ├── __init__.py
│       └── model_service.py
├── requirements.txt
├── .env
└── README.md
```

Tech Stack

Python · FastAPI · Pydantic · Uvicorn · OpenAI API · LangChain · PostgreSQL · MongoDB · Docker

Install Dependencies

```
# Create virtual environment
python -m venv venv

# Activate (Windows)
venv\Scripts\activate

# Activate (Mac/Linux)
source venv/bin/activate

# Install required packages
pip install fastapi uvicorn python-dotenv
pip install pydantic
pip install openai langchain
pip install sqlalchemy psycopg2-binary
pip install motor  # for MongoDB (optional)
pip install python-multipart  # for file uploads
```

Hello World with FastAPI

```
# main.py
from fastapi import FastAPI
app = FastAPI(title="AI API", version="1.0")

@app.get("/")
def read_root():
    return {"message": "Hello from FastAPI!"}

@app.get("/health")
def health_check():
    return {"status": "healthy"}
```

Run the Server

```
# Run using Uvicorn
uvicorn app.main:app --reload
```

Server will start at:
http://127.0.0.1:8000

Docs available at:
http://127.0.0.1:8000/docs
(Swagger UI)

Pydantic Models (Request / Response)

```
# models/schemas.py
from pydantic import BaseModel

class ChatRequest(BaseModel):
    message: str
    model: str = "gpt-3.5-turbo"

class ChatResponse(BaseModel):
    response: str
    model: str
    tokens: int
```

AI Endpoint with OpenAI API

```
# routes/ai_routes.py
from fastapi import APIRouter
from models.schemas import ChatRequest, ChatResponse
import openai, os

router = APIRouter()
openai.api_key = os.getenv("OPENAI_API_KEY")

@router.post("/chat", response_model=ChatResponse)
def chat_with_ai(request: ChatRequest):
    response = openai.ChatCompletion.create(
        model=request.model,
        messages=[{"role": "user", "content": request.message}]
    )
    return ChatResponse(
        response=response.choices[0].message.content,
        model=request.model,
    ) tokens=response.usage.total_tokens
```

Include Router in Main

```
# main.py
from fastapi import FastAPI
from routes.ai_routes import router

app = FastAPI(title="AI API")

app.include_router(router, prefix="/api")

@app.get("/")
def read_root():
    return {"message": "AI API is running"}
```

Test the API

Request (POST /api/chat)

```
{
  "message": "Explain FastAPI in simple words",
  "model": "gpt-3.5-turbo"
}
```

Response

```
{
  "response": "FastAPI is a modern Python
   framework for building APIs ...",
  "model": "gpt-3.5-turbo",
  "tokens": 120
}
```

File Upload (Images / Documents)

```
from fastapi import UploadFile, File

@app.post("/upload")
async def upload_file(file: UploadFile = File(...)):
    contents = await file.read()
    return {
      "filename": file.filename,
      "size": len(contents),
      "message": "File uploaded successfully"
    }
```

Environment Variables

```
# .env
OPENAI_API_KEY=your_openai_key
DATABASE_URL=postgresql://user:pass@localhost/db
MONGO_URL=mongodb://localhost:27017/aiapp
```

```
# Load in Python
from dotenv import load_dotenv
import os
load_dotenv()
openai.api_key = os.getenv("OPENAI_API_KEY")
```

Connecting to Database (Optional)

PostgreSQL (SQLAlchemy)

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

DATABASE_URL = os.getenv("DATABASE_URL")
engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(
  autocommit=False, autoflush=False,
  bind=engine
)
```

MongoDB (Motor)

```
from motor.motor_asyncio import
    AsyncIOMotorClient

MONGO_URL = os.getenv("MONGO_URL")

client = AsyncIOMotorClient(MONGO_URL)
db = client["aiapp"]
collection = db["conversations"]
```

Real World AI Application Ideas

AI Chatbot API · Document / PDF Q&A API · Image Analysis API · Resume Analyzer · Text Summarizer

Best Practices

- Use environment variables for API keys
- Validate input using Pydantic models
- Handle errors properly
- Use async for better performance
- Keep API modular (routers, services, models)
- Add logging and monitoring
- Use Docker for deployment
- Follow security best practices

Useful Tools

Swagger UI (FastAPI Docs) · Postman (API Testing) · Docker (Containerization) · VS Code (Development)

6. SQL + PostgreSQL + MongoDB

@codedsoul_05

What is SQL ?

- SQL (Structured Query Language) is used to manage and manipulate relational databases.
- It is used to create, read, update and delete data.
- It works with tables, rows and columns.
- It is widely used in web applications, analytics and business applications.

What is PostgreSQL ?

- PostgreSQL is an open source relational database management system (RDBMS).
- Supports advanced features like JSONB, full text search, indexing, transactions and more.
- It is highly reliable, scalable and widely used in production.

What is MongoDB ?

- MongoDB is a NoSQL database that stores data in flexible JSON like documents.
- It is schema less and easy to scale.
- It is widely used for web applications, real time applications and AI projects.

SQL Basic Syntax

```sql
-- Create table
CREATE TABLE users (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    age INT,
    email VARCHAR(100)
);
-- Insert data
INSERT INTO users (id, name, age, email)
VALUES (1, 'Nikhil', 24, 'nikhil@gmail.com');

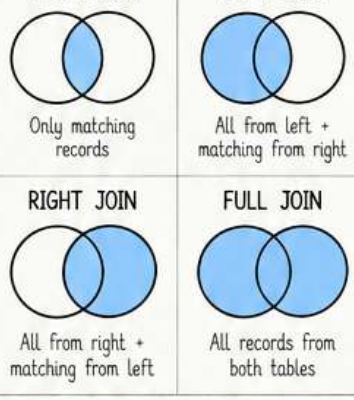
-- Select data
SELECT * FROM users;
SELECT name, age FROM users WHERE age > 20;

-- Update data
UPDATE users SET age = 25 WHERE id = 1;
-- Delete data
DELETE FROM users WHERE id = 1;
```

SQL Commands

Category	Commands	Example
Create	CREATE	CREATE TABLE users (...);
Read	SELECT	SELECT * FROM users;
Update	UPDATE	UPDATE users SET age=25;
Delete	DELETE	DELETE FROM users WHERE id=1;
Filter	WHERE	SELECT * FROM users WHERE age=20;
Sort	ORDER BY	SELECT * FROM users ORDER BY name;
Limit	LIMIT	SELECT * FROM users LIMIT 10;
Group	GROUP BY	SELECT age, COUNT(*) FROM users GROUP BY age;
Join	JOIN	SELECT * FROM a JOIN b ON a.id=b.id;
Aggregate	COUNT, SUM, AVG, MAX, MIN	SELECT COUNT(*) FROM users;

SQL Joins



PostgreSQL Features

- ACID compliance (reliable transactions)
- Supports advanced data types (JSONB, Arrays, UUID)
- Full text search and indexing
- Window functions and CTEs
- Extensible (custom functions, extensions)
- Strong community support

PostgreSQL Example

```sql
-- Create table
CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    price DECIMAL(10,2),
    created_at TIMESTAMP DEFAULT NOW()
);
-- Insert
INSERT INTO products (name, price)
VALUES ('Laptop', 55000.00);

-- JSONB example
SELECT * FROM products
WHERE attributes->>''brand' = 'Dell';
```

PostgreSQL JSONB Example

```sql
-- Create table with JSONB column
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    profile JSONB
);

-- Insert JSON data
INSERT INTO users (name, profile)
VALUES ('Nikhil', '{"age":24, "city":"Ahmedabad"}');

-- Query JSON data
SELECT profile->>'city' AS city
FROM users
WHERE profile->>'age' = '24';
```

MongoDB Features

- Document based (JSON like format)
- Schema less and flexible
- Easy horizontal scaling
- High performance for large data
- Supports real time applications
- Widely used in web and AI applications

MongoDB Basic Operations

```javascript
// Insert document
use mydb
db.users.insertOne({
  name: "Nikhil",
  age: 24,
  city: "Ahmedabad"
})

// Find documents
db.users.find({age: { $gt: 20
  } })

// Update document
db.users.updatmen(.,
db.users.updateOne(
  { name: "Nikhil" },
  { $set: { age: 25 } }
)

// Delete document
db.users.deleteOne({
  { name: "Nikhil" })
```

MongoDB Data Model

```javascript
{
  "_id": ObjectId("64f7b2e9c1a2b3c4d5e6f7a8"),
  "name": "Nikhil",
  "age": 24,
  "email": "nikhil@gmail.com",
  "skills": ["JavaScript", "Python", "React"],
  "address": {
    "city": "Ahmedabad",
    "country": "India"
  },
  "isActive": true,
  "createdAt": ISODate("2026-09-21T10:00:00Z")
}
```

SQL vs NoSQL Comparison

Feature	SQL (PostgreSQL)	NoSQL (MongoDB)
Data Model	Tables (rows, columns)	Documents (JSON)
Schema	Fixed schema	Schema less
Query Language	SQL	MongoDB Query API
Scalability	Vertical	Horizontal
Transactions	Full ACID support	Limited (multi document in recent versions)
Best For	Structured data, complex queries	Flexible data, real time apps, large scale applications

When to Use What ?

Use PostgreSQL when:

- You need structured data
- You need complex queries and joins
- You need transactions and reliability
- You are building enterprise applications
- You need advanced features (JSONB, full text search)

Use MongoDB when:

- You need flexible and dynamic schema
- You are working with large scale data
- You need fast development and scaling
- You are building real time or content heavy applications
- You are working with AI/ML data (logs, embeddings, documents)

Real World Use Cases

PostgreSQL

- User management systems
- E-commerce platforms
- Analytics and reporting
- Financial and banking systems
- Applications with complex relationships
- Storing structured AI/ML metadata

MongoDB

- Content management systems
- Real time chat applications
- E-commerce product catalogs
- Storing AI embeddings and documents
- IoT and sensor data
- Applications with frequently changing data

Best Practices

- Use proper indexing for better performance
- Use meaningful table and field names
- Follow normalization in SQL
- Use transactions for critical operations
- Keep data backup and monitoring
- Use environment variables for database credentials
- Optimize queries and avoid N+1 queries
- Use connection pooling in production

Useful Tools



Sample Project Ideas

- User management system (PostgreSQL)
- Blog application with comments (PostgreSQL / MongoDB)
- E commerce product catalog (MongoDB)
- AI document storage system (MongoDB)
- Analytics dashboard (PostgreSQL)
- Chat application (MongoDB)

7. Authentication + Authorization + API Security

@codedsoul_05

What is Authentication ?

- Authentication is the process of verifying who a user is.
- It checks the identity of the user using credentials like username, password, OTP, OAuth, etc.
- After successful authentication, a session or token is created.

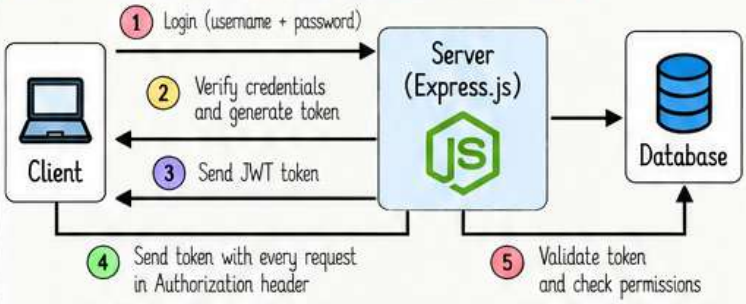
What is Authorization ?

- Authorization is the process of checking what a user is allowed to do.
- It decides the permissions and access level of the authenticated user.
- Example: admin can delete, user can only read.

What is API Security ?

- API Security protects APIs from unauthorized access, attacks and misuse.
- It ensures only valid and trusted clients can access the API.
- It includes authentication, authorization, input validation, rate limiting, encryption, CORS, and more.

How Authentication & Authorization Work ?



Common Authentication Methods

Method	Description	Use Case
Username + Password	Basic authentication	Simple applications
JWT (JSON Web Token)	Token based authentication	Web and mobile apps
OAuth 2.0	Third party authentication (Google, GitHub, etc.)	Social login, enterprise apps
API Key	Unique key for accessing API	External APIs, microservices
Session Based	Server side session storage	Traditional web applications
OTP / 2FA	Additional verification step	High security applications
Biometric	Fingerprint / Face ID	Mobile applications

Password Hashing (Bcrypt)

```
// Install
npm install bcryptjs

// Hash password
const bcrypt = require('bcryptjs');

async function hashPassword(password) {
  const salt = await bcrypt.genSalt(10);
  const hashed = await bcrypt.hash(password, salt);
  return hashed;
}

// Compare password
async function comparePassword(password, hashed) {
  return await bcrypt.compare(password, hashed);
}
```

JWT (JSON Web Token)

```
// Install
npm install jsonwebtoben
const jwt = require('jsonwkotoken');
const JWT_SECRET = "your_secret_key";

// Generate token
function generateToken(user) {
  return jwt.sign(
    { id: user.id, role: user.role },
    JWT_SECRET,
    { expiresIn: "1d" }
  );
};

// Verify token
function verifyToken(token) {
  return jwt.verify(token, JWT_SECRET);
```

Middleware for Protected Routes

```
const jwt = require('jsonwkotoken');
const JWT_SECRET = "your_secret_key";

function authenticateToken(req, res, next) {
  const token = req.haeders['authorization']?.split(' '){1};

  if (!token) {
    return res.status(401).json({ message: "Access denied" });
  }
  try {
    const decoded = jwt.verify(token, JWT_SECRET);
    req.user = decoded; // attach user info
    next();
  } c atch (error) {
    return res.status(403).json({ message: "Invalid token" });
  }
module.exports = authenticateToken;
```

Role Based Authorization (RBAC)

```
function authorizeRole(roles) {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({
        message: "You do not have permission"
      });
    }
    next();
  };
// Example usage
app.get('/admin', authenticateToken,
  authorizeRole(['admin']), (req, res) => {
  res.json({ message: "Admin access granted" });
```

OAuth 2.0 (Google Login Example)

```
// Install
npm install passport passport-google-oauth20

const passport = require('passport');
const GoogleStrategy = require(
 'passport-poogle-oauth20').Strategy;

passport.use(new GoogleStrategy({
  clientID: "YOUR_CLIENT_ID",
  clientSecret: "YOUR_CLIENT_SECRET",
  callbackURL: "/auth/google/callback"
}, (accessToken, refreshToken, profile, done) => {
  // Find or create user in DB
  return done(null, profile);
}));
```

API Key Authentication

```
const API_KEYS = ["key1", "key2", "key3"];

function apiKeyAuth(req, res, next) {
  const key = req.headers['x-api-key"];
  if (!key || !API_KEYS.includes(key)) {
    return res.status(401).json({
      message: "Invalid API Key"
    });
  }
  next();

app.use('/api', apiKeyAuth);

app.get('/api/data', (req, res) => {
  res.json({ message: "Public data" });
```

API Security Best Practices

- Use HTTPS (always)
- Validate and sanitize all inputs
- Use strong authentication (JWT, OAuth, etc.)
- Implement proper authorization (RBAC)
- Use rate limiting to prevent abuse
- Implement CORS (Cross Origin Resource Sharing)
- Use API keys for external access
- Handle errors properly (no sensitive data in responses)
- Use environment variables for secrets
- Enable logging and monitoring
- Use request size limits and input validation
- Follow principle of least privilege

Common Security Attacks

Attack	Description
Brute Force	Trying multiple passwords to gain access
SQL Injection	Injecting malicious SQL queries
XSS (Cross Site Scripting)	Injecting malicious scripts
CSRF (Cross Site Request Forgery)	Forcing authenticated users to perform unwanted actions
DDoS	Overloading the server with requests
Man in the Middle	Intercepting data between client and server
Token Theft	Stealing JWT or session tokens
Broken Access Control	Accessing resources without permission
Insecure APIs	Exposing sensitive data or endpoints

Rate Limiting (express-rate-limit)

```
// Install
npm install express-rate-limit

const rateLimit = require('express-rate-limit');

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // limit each IP to 100 requests
  message: "Too many requests, try again later"
});

app.use('/api', limiter);
```

Environment Variables (.env)

```
STORE SENSITIVE DATA IN .ENV FILE

JWT_SECRET=your_secret_key
DB_URL=your_database_url
GOOGLE_CLIENT_ID=your_client_id
GOOGLE_CLIENT_SECRET=your_client_secret
API_KEY=your_api_key
```

CORS (Cross Origin Resource Sharing)

```
const cors = require('cors');

app.use(cors({
  origin: 'https://your-frontend.com',
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  credentials: true
}));
```

Secure API Response Example

```
res.json({
  success: true,
  message: "User logged in successfully",
  data: {
    id: user.id,
    name: user.name,
    role: user.role
});
```

Useful Tools

- Postman (API Testing)
- JWT.io (Token Debugger)
- OWASP ZAP (Security Testing)
- Burp Suite (Penetration Testing)
- Nmap (Network Scanning)
- Helmet.js (Security Headers)

8. AI + Machine Learning Fundamentals

@codedsoul_05

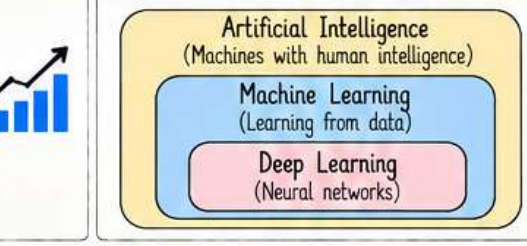
What is AI ?

- AI (Artificial Intelligence) is the simulation of human intelligence in machines.
- It can understand, learn, reason, solve problems and make decisions.
- Examples: ChatGPT, self driving cars, recommendation systems, virtual assistants.

What is Machine Learning ?

- Machine Learning is a subset of AI that enables systems to learn from data without being explicitly programmed.
- It uses algorithms to find patterns in data and make predictions.
- Examples: spam detection, image classification, price prediction.

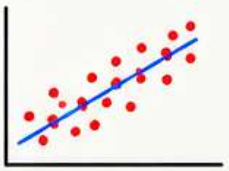
AI vs ML vs Deep Learning



Types of Machine Learning

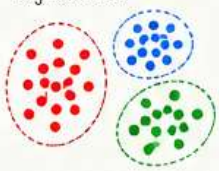
1. Supervised Learning

- Learns from labeled data (input → output).
- Used for classification and regression.
- Examples: house price prediction, spam detection.



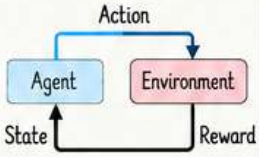
2. Unsupervised Learning

- Learns from unlabeled data.
- Finds hidden patterns and structures.
- Used for clustering and dimensionality reduction.
- Examples: customer segmentation.

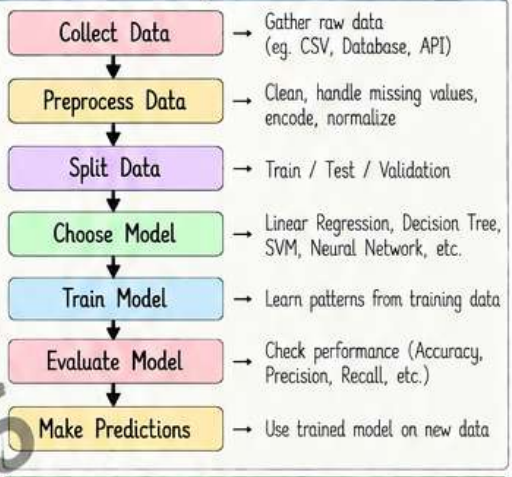


3. Reinforcement Learning

- Learns by interacting with an environment.
- Receives rewards or penalties.
- Used in robotics, gaming and autonomous systems.



Machine Learning Workflow



Popular ML Algorithms

Category	Algorithms
Regression	Linear Regression, Ridge, Lasso
Classification	Logistic Regression, Decision Tree, Random Forest, SVM, Naive Bayes
Clustering	K Means, Hierarchical Clustering, DBSCAN
Dimensionality Reduction	PCA, t SNE
Ensemble Methods	Random Forest, Gradient Boosting, XGBoost
Neural Networks	ANN, CNN, RNN, Transformers

Key Terminologies

Term	Meaning
Dataset	Collection of data (rows and columns)
Feature	Input variable (independent variable)
Label / Target	Output variable (dependent variable)
Model	Learns pattern from data
Training	Process of learning from data
Testing	Checking performance on new data
Overfitting	Good on training data but poor on new data
Underfitting	Fails to learn pattern properly
Accuracy	Correct predictions / total predictions
Precision	Correct positive predictions / all positives
Recall	Correct positive predictions / actual positives
F1 Score	Harmonic mean of precision and recall

@codedsoul_05

Evaluation Metrics

Confusion Matrix

Actual \ Predicted	Positive	Negative
Positive (True)	TP	FN
Negative (False)	FP	TN

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Linear Regression Example (Python)

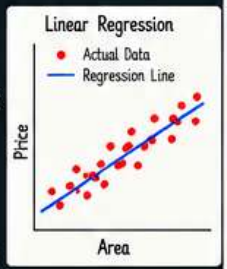
```python
# Predicting house price using Linear Regression
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

df = pd.read_csv("house_data.csv")
X = df[["area", "bedrooms", "age"]]
y = df["price"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

pred = model.predict(X_test)
print("MSE:", mean_squared_error(y_test, pred))
```



Classification Example (Python)

```python
# Spam detection using Logistic Regression
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

df = pd.read_csv("emails.csv")
X = df["text"]
y = df["label"]   # 0 = not spam, 1 = spam

vectorizer = CountVectorizer()
X_vec = vectorizer.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_vec, y, test_size=0.2, random_state=42)

model = LogisticRegression()
model.fit(X_train, y_train)

pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, pred))
```

Clustering Example (Python)

```python
# Customer segmentation using K Means
import pandas as pd
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

df = pd.read_csv("customers.csv")
X = df[["age", "income"]]

kmeans = KMeans(n_clusters=3, random_state=42)
df["cluster"] = kmeans.fit_predict(X)

plt.scatter(df["age"], df["income"],
            c=df["cluster"], cmap="viridis")
plt.xlabel("Age")
plt.ylabel("Income")
plt.title("Customer Segmentation (K Means)")
plt.show()
```

Common ML Libraries



Real World Applications



When to Use Which Algorithm ?

Problem Type	Suggested Algorithms
Predicting continuous values	Linear Regression, Random Forest Regression
Classifying categories	Logistic Regression, SVM, Random Forest
Grouping similar data	K Means, Hierarchical Clustering
Reducing dimensions	PCA, t SNE
Working with images/text	Neural Networks, CNN, RNN, Transformers

Best Practices

- ✅ Always clean and preprocess data
- ✅ Use train-test split and cross validation
- ✅ Try multiple algorithms and compare performance
- ✅ Handle overfitting (regularization, dropout, early stopping)
- ✅ Use proper evaluation metrics (Accuracy, Precision, Recall, F1)
- ✅ Feature engineering is very important
- ✅ Visualize data for better understanding
- ✅ Keep model simple and interpretable

Limitations of Machine Learning

- ❌ Requires good quality and sufficient data
- ❌ Performance depends on data preprocessing
- ❌ Can be biased if data is biased
- ❌ Does not work well with completely new situations
- ❌ Not easily interpretable (especially deep learning)
- ❌ Requires continuous monitoring and retraining
- ❌ Cannot replace human intelligence fully

9. LLMs + Generative AI + Prompt Engineering

@codedsoul_05

What are LLMs ?

- LLMs (Large Language Models) are AI models trained on massive text data to understand and generate human like text.
- They can answer questions, summarize content, write code, and perform different tasks.
- Examples: GPT 4, Llama, Mistral, Claude, Gemini, etc.

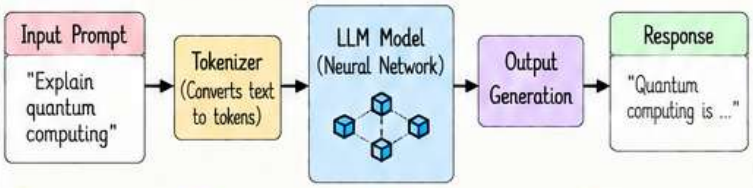
What is Generative AI ?

- Generative AI refers to models that can create new content like text, images, audio, video and code.
- It learns patterns from existing data and generates similar but new content.
- Examples: ChatGPT (text), DALL·E (images), Sora (video), GitHub Copilot (code).

Why is Prompt Engineering Important ?

- Quality of output depends on the prompt
- Helps to get accurate and relevant responses
- Reduces hallucinations (wrong answers)
- Useful for complex tasks like coding, analysis, summarization
- Essential for building AI applications and agents

How LLMs Work ? (High Level)



1. **Tokenize** — Input text is converted into tokens (numbers).
2. **Process** — Model predicts next token based on context.
3. **Generate** — Tokens are converted back to text.
4. **Response** — Final output is generated based on the prompt.

Popular LLM Models

Model	Company	Strengths	Use Case
GPT 4	OpenAI	High quality, multi modal	Chat, coding, analysis
GPT 4o	OpenAI	Fast, multi modal (text, image, audio)	Real time applications
Claude 3	Anthropic	Safe, long context	Research, analysis
Llama 3	Meta	Open source, customizable	Fine tuning, local use
Gemini	Google	Multi modal, strong tools integration	Search, productivity
Mistral	Mistral AI	Efficient, open source	Deployment, custom apps

Prompt Engineering Techniques

1. **Zero Shot Prompting** — Directly ask without examples.
2. **Few Shot Prompting** — Provide some examples in the prompt.
3. **Chain of Thought** — Ask the model to think step by step.
4. **Role Prompting** — Give a specific role to the AI (e.g. You are a teacher).
5. **Structured Prompting** — Use clear format and instructions.
6. **Iterative Prompting** — Refine and improve the prompt based on output.

Prompt Examples

1. Zero Shot Prompt

```
"Explain blockchain in simple words."
```

2. Few Shot Prompt

```
"Translate the following:n
  English: Hello   Hindi: Namaste
  English: Thank you  Hindi: Dhanyavaad
  English: Good morning   Hindi:"
```

3. Chain of Thought Prompt

```
""Solve this math problem step by step:
  If a train travels 120 km in 2 hours,
  what is its speed?"
```

4. Role Prompt

```
""You are an expert software engineer.
  Explain the difference between SQL
  and NoSQL in simple words with examples."
```

5. Structured Prompt

```
"Summarize the given article in 5 bullet points:
 Text: <your text here>
 Format:
  - Point 1
  - Point 2
  - Point 3
  - Point 4
  - Point 5"
```

6. Iterative Prompt

```
 "The previous answer missed some points.
  Now include advantages and disadvantages also."
```

@Codedsoul_05

Using OpenAI API (Python)

```python
# Install
pip install openai

# Code
from openai import OpenAI
client = OpenAI(api_key="YOUR_API_KEY")

response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
      {"role": "user",
       "content": "Explain quantum
          computing in simple words."}
    ],
    temperature=0.7
)
print(response.choices[0].message.content)
```

Controlling Output

```python
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": "Write
    a short story about a robot."}],
    temperature=0.7,  # creativity (0 to 1)
    max_tokens=200,   # limit length
    top_p=0.9,        # alternative to temperature
    frequency_penalty=0.2, # reduce repetition
    presence_penalty=0.2   # encourage new topics
)
```

Other Generative AI Models

Image Generation (DALL·E)

```python
from openai import OpenAI
client = OpenAI(api_key="YOUR_API_KEY")
response = client.images.generate(
    model="dall-e-3",
    prompt="A futuristic city at night",
    n=1, size="1024x1024"
)
```

Text to Speech (TTS)

```python
from openai import OpenAI
client = OpenAI(api_key="YOUR_API_KEY")
response = client.audio.speech.create(
  model="tts-1", input="Hello, this is AI speaking.")
```

Video Generation (Sora)

Sora can generate videos from text prompts (available through OpenAI platform).

Common Prompting Use Cases

Content Creation · Summarization · Code Generation · Data Analysis · Chatbots · Image Generation · Automation

Prompting Best Practices

- Be clear and specific in your instructions
- Provide context and constraints
- Use examples where needed
- Break complex tasks into smaller steps
- Specify the desired format (e.g. JSON, table, bullet points)
- Iterate and refine the prompt
- Validate the output and handle errors
- Use system prompts for consistent behavior

Limitations of LLMs

- Can generate incorrect or biased information (hallucinations)
- Limited real time knowledge (depends on training data)
- Struggles with very complex reasoning
- Output quality depends heavily on the prompt
- Not always up to date
- May produce harmful or inappropriate content

Real World Applications

Virtual Assistants · AI Search Engines · Education & Learning · Healthcare Support · Marketing & Content · Software Development · Creative Design

10. OpenAI APIs + AI Integration

@Codedsoul_05

What are OpenAI APIs ?

- OpenAI provides APIs to access powerful AI models like GPT-4, GPT-4o, DALL·E, Whisper, and more.
- You can integrate these models into your own applications using simple HTTP requests.
- Used for chat, text generation, image generation, speech to text, embeddings, and more.
- Helps you build AI powered features easily without training your own model.

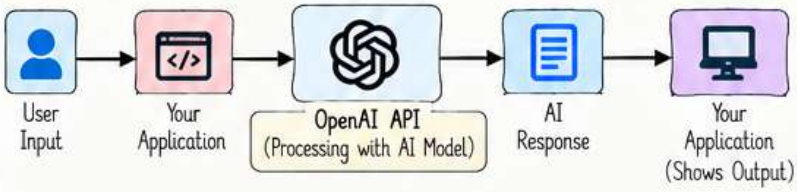
Popular OpenAI Models

Model	Use Case
gpt-4o	Latest multi modal model (text, image, audio)
gpt-4-turbo	High performance text model
gpt-3.5-turbo	Fast and cost effective model
dall-e-3	Image generation from text
whisper-1	Speech to text (audio transcription)
tts-1	Text to speech
text-embedding-3	Generate embeddings for RAG and semantic search

Why Use OpenAI APIs ?

- Easy to integrate with your application
- High quality results for text, image and audio
- Saves time and resources (no model training)
- Regularly updated with latest capabilities
- Supports real world use cases like chatbots, content generation, image creation, and more
- Can be combined with your own data (knowledge base) for custom solutions

How OpenAI API Works ?



Authentication (API Key)

1. Create an account at https://platform.openai.com
2. Generate your API key from dashboard
3. Keep it secure and never expose it in frontend
4. Use it in your application via environment variables

```
# .env file
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxx
```

1. Chat Completion (Text Generation)

```python
from openai import OpenAI
import os

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Explain machine learning in simple words."}
    ],
    temperature=0.7
)
print(response.choices[0].message.content)
```

2. Image Generation (DALL·E)

```python
from openai import OpenAI
import os

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

response = client.images.generate(
    model="dall-e-3",
    prompt="A modern workspace with a laptop and plants, realistic style",
    n=1,
    size="1024x1024"
)
image_url = response.data[0].url
print(image_url)
```



3. Speech to Text (Whisper)

```python
from openai import OpenAI

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

with open("audio.mp3", "rb") as audio_file:
  transcript = client.audio.transcriptions.create(
    model="whisper-1",
    file=audio_file,
    response_format="text"
  )

print(transcript)
```

4. Text to Speech (TTS)

```python
from openai import OpenAI

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

response = client.audio.speech.create(
    model="tts-1",
    voice="alloy",
    input="Welcome to my AI application."
)

with open("output.mp3", "wb") as f:
  f.write(response.content)
```

5. Embeddings (For RAG / Semantic Search)

```python
from openai import OpenAI

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

response = client.embeddings.create(
    model="text-embedding-3-small",
    input="OpenAI makes powerful AI models."
)

embedding = response.data[0].embedding
print(len(embedding))  # 1536 dimensions
```

```
[ 0.1, 0.2, 0.3, ... ]
```

6. Function Calling (Structured Output)

```python
from openai import OpenAI
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "user", "content": "Get weather in Ahmedabaad"}
    ],
    tools=[{
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "Get current weather",
            "parameters": {"type": "object", ...}
    }} ]
print(response.choices[0].message)
```

Integration with Web Applications

@Codedsoul_05

Frontend (React / Next.js)

- Send user input
- Show AI response
- Handle file uploads

Backend (Node.js / Python)

- Call OpenAI API
- Process response
- Add business logic
- Use environment variables

(Database (Optional)

- Store chat history
- Store embeddings
- Build RAG applications

Example: AI Chatbot with Next.js (API Route)

```typescript
// app/api/chat/route.ts
import { OpenAI } from "openai";
const client = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });

export async function POST(req: Request) {
  const { message } = await req.json();

  const response = await client.chat.completions.create({
    model: "gpt-4o",
    messages: [{ role: "user", content: message }],
  });

  return Response.json({
    reply: response.choices[0].message.content,
  });
}
```

Real World Use Cases

- AI Chatbots (Customer Support)
- Content Generation
- Image Creation
- Document Summarization
- Code Assistant
- Voice Applications
- RAG (Knowledge Base)

Best Practices

- Use environment variables for API keys
- Validate and sanitize user inputs
- Handle errors and rate limits gracefully
- Use appropriate model for your use case
- Implement caching for repeated queries
- Keep prompts clear and specific
- Monitor token usage and costs
- Follow OpenAI usage policies

Important Tools & Libraries

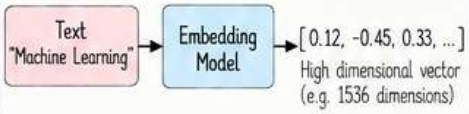
- OpenAI SDK
- LangChain
- Vercel (Deployment)
- Postman (API Testing)
- Zod (Validation)
- Pinecone (Vector DB)

11. Embeddings + Vector Databases + RAG

@Codedsoul_05

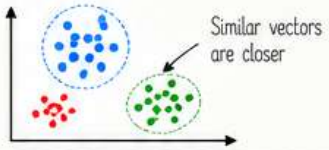
What are Embeddings ?

- Embeddings convert text, images, or other data into numerical vectors.
- These vectors capture the meaning and context of the data.
- Similar items have similar vectors (closer in vector space).
- Used for search, clustering, recommendation, and RAG.



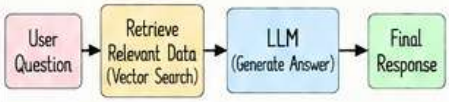
What are Vector Databases ?

- Vector databases store and search embeddings efficiently.
- They use similarity search (e.g. cosine similarity) instead of exact match.
- Designed for large scale unstructured data (text, images, audio, etc.).
- Examples: Pinecone, Weaviate, Qdrant, Chroma, Milvus, Faiss, MongoDB Atlas Vector Search.



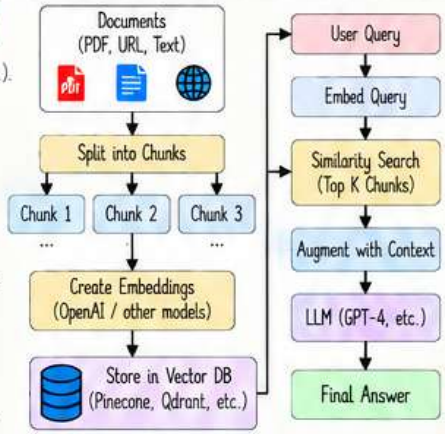
What is RAG ?

- RAG (Retrieval-Augmented Generation) combines retrieval (search) with generation (LLM).
- Instead of relying only on the model's training data, it retrieves relevant documents from your own data source.
- Helps in accurate, up-to-date, and domain-specific answers.
- Used in chatbots, document Q&A, enterprise search, and knowledge bases.



How RAG Works ?

1. **Ingest Data:** Load your documents (PDF, text, website, etc.).
2. **Split Documents:** Break into smaller chunks.
3. **Create Embeddings:** Convert each chunk into vectors.
4. **Store in Vector DB:** Save embeddings with metadata.
5. **Retrieve:** When a user asks a question, find the most relevant chunks using similarity search.
6. **Augment:** Provide retrieved context to the LLM.
7. **Generate:** LLM generates the final answer using context.



Embedding Models

Model	Provider	Dimensions	Use Case
text-embedding-3-small	OpenAI	1536	General purpose, fast, cost effective
text-embedding-3-large	OpenAI	3072	High quality, better accuracy
all-MiniLM-L6-v2	SentenceTransformers	384	Open source, lightweight
bge-base-en	BAAI	768	Good for semantic search
e5-large	Meta	1024	Strong for multilingual search

Generating Embeddings (Python)

```python
from openai import OpenAI
client = OpenAI(api_key="YOUR_API_KEY")

text = "Machine Learning is a subset of AI."
response = client.embeddings.create(
    model="text-embedding-3-small",
    input=text
)
embedding = response.data[0].embedding
print(len(embedding))  # 1536
```

Storing in Vector Database (Pinecone Example)

```python
from pinecone import Pinecone
from openai import OpenAI

pc = Pinecone(api_key="YOUR_PINECONE_KEY")
index = pc.Index("rag-index")

client = OpenAI(api_key="YOUR_OPENAI_KEY")
text = "This is a sample document."
emb = client.embeddings.create(
    model="text-embedding-3-small",
    input=text
).data[0].embedding

index.upsert(
   vectors=[{"id": "doc1", "values": emb,
)          "metadata": {"text": text}}]
```

Similarity Search (Retrieve Relevant Data)

```python
query = "What is machine learning?"
query_emb = client.embeddings.create(
    model="text-embedding-3-small",
    input=query
).data[0].embedding

results = index.query(
   vector=query_emb,
   top_k=3,
   include_metadata=True
)
for match in results['matches']:
  print(match['score'], match['metadata']['text'])
```

RAG with OpenAI (Complete Example)

```python
# 1. Retrieve relevant context
context = "".join([m['metadata']['text']
                   for m in results['matches']])

# 2. Create prompt
prompt = f"""
You are a helpful assistant.
Use the following context to answer the question.

Context:
{context}

Question: {query}
Answer:
"""
response = client.chat.completions.create(
    model="gpt-4o",
    messages=["role": "user", "content": prompt}]
)
print(response.choices[0].message.content)
```

Popular Vector Databases

Database	Type	Key Features	Use Case
Pinecone	Cloud	Fully managed, high performance	Production apps, scalable RAG
Weaviate	open source	Graph + vector hybrid search	Flexible, open source
Qdrant	Open source	Fast, easy to use	Real time search
Chroma	open source	Simple and lightweight	Small to medium projects
Milvus	open source	High scalability	Large scale apps
MongoDB Atlas	Cloud	Vector search with existing data	Combine structured + unstructured data
Faiss	open source	Efficient similarity search	Research, custom apps

RAG Chunking Strategies

- Split text into small, meaningful chunks (e.g. 500-1000 tokens).
- Use overlap between chunks (e.g. 100 tokens).
- Keep contextually related information together.
- Clean and preprosecs text before embedding.
- Store metadata (source, page number, etc.).

Evaluation Metrics

- **Relevance:** Are retrieved chunks relevant ?
- **Accuracy:** Does the answer match the source ?
- **Faithfulness:** Is the model using the context or hallucinating ?
- **Latency:** Response time
- **Cost:** API and storage cost

Real World Use Cases

- Chatbots (Customer Support)
- Document Q&A (Internal Knowledge)
- Enterprise Search (Company Data)
- Research Assistant (Papers, Books)
- E-commerce Search (Product Recommendations)
- Healthcare (Medical Docs)

Best Practices

- Use high quality embedding models.
- Choose right chunk size and overlap.
- Store metadata for better filtering.
- Use hybrid search (keyword + vector) if needed.
- Keep data updated and re-embed when changed.
- Handle out of scope queries gracefully.
- Monitor retrieval quality and usage cost.

Common Challenges

- Poor chunking leads to wrong answers.
- Including irrelevant data in context.
- Model hallucination despite retrieved data.
- High storage and API costs.
- Latency in large datasets.
- Keeping knowledge base updated.
- Handling multilingual data.

12. LangChain + LangGraph + AI Agents

@Codedsoul_05

What is LangChain ?

- LangChain is an open source framework to build applications using LLMs.
- It provides tools, chains, memory, retrievers, agents and integrations with different data sources.
- Helps you build real world AI applications like chatbots, document Q&A, agents, etc.

LangChain

What is LangGraph ?

- LangGraph is a framework (by LangChain) for building stateful, multi-step, and cyclic AI workflows.
- It represents the flow as a graph nodes and edges.
- Useful for complex agentic workflows with decision making, tool use, loops and human-in-the-loop.

LangGraph

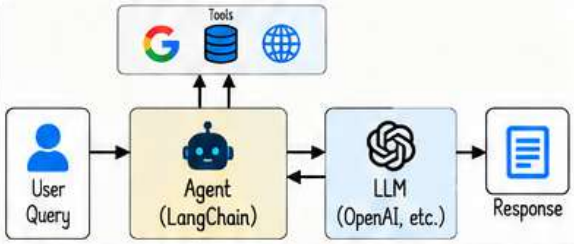
What are AI Agents ?

- AI Agents are systems that can perceive, plan, take actions and use tools to achieve a goal.
- They can autonomously decide which tools to use and in what order.
- Examples: research agents, coding agents, data analysis agents, support agents, etc.

Key Components in LangChain

LLM	Prompts	Chains	Memory	Tools	Agents
ChatOpenAI OpenAI HuggingFace	PromptTemplate ChatPromptTemplate	Combine steps together	Store conversation history	Use external tools (APIs, DB, Search, Python, etc.)	Decide which tool to use and execute

How It All Works Together ?



Simple LangChain Example (LLM Chain)

```
# Install
pip install langchain langchain-openai

from langchain_openai import ChatOpenAI
from langchain.prompts import ChatPromptTemplate
from langchain.chains import LLMChain

llm = ChatOpenAI(model="gpt-4o",
                 api_key="YOUR_API_KEY")

prompt = ChatPromptTemplate.from_template(
    "Explain {topic} in simple words."
)

chain = LLMChain(llm=llm, prompt=prompt)

response = chain.run({"topic": "vector databases"})
print(response)
```

Agent with Tools (LangChain)

```
from langchain.agents import initialize_agent,
     Tool, AgentType
from langchain_openai import ChatOpenAI

# Define a tool (web search example)
def search_tool(query: str) -> str:
    # use any search API (e.g. Tavily, SerpAPI)
    return "Search result for: " + query

tools = [
    Tool(
        name="Search",
        func=search_tool,
        description="Useful for searching
          latest information from the web"
    )
]

llm = ChatOpenAI(model="gpt-4o",api_key="YOUR_API_KEY")
agent = initialize_agent(tools, llm,
            agent=AgentType.OPENAI_FUNCTIONS,
            verbose=True)

agent.run("What are the latest AI news today?")
```

LangGraph Workflow (Agent with Flow)

```
from langgraph.graph import StateGraph, END
from typing import TypedDict

class AgentState(TypedDict):
    question: str
    answer: str
    steps: list

workflow = StateGraph(AgentState)
# Define nodes
def plan_node(state):
    return {"steps": state["steps"] + ["plan"]}
def tool_node(state):
    return {"steps": state["steps"] + ["tool"]}
def answer_node(state):
    return {"answer": "Final answer",
            "steps": state["steps"] + ["answer"]}

workflow.add_node("plan", plan_node)
workflow.add_node("tool", tool_node)
workflow.add_node("answer", answer_node)
workflow.set_entry_point("plan")
workflow.add_edge("plan", "tool")
workflow.add_edge("tool", "answer")
workflow.add_edge("answer", END)

app = workflow.compile()
result = app.invoke({"question": "Solve using tools"})
print(result)
```

RAG with LangChain (Complete Example)

```
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Pinecone
from langchain.chains import RetrievalQA
from langchain_openai import ChatOpenAI

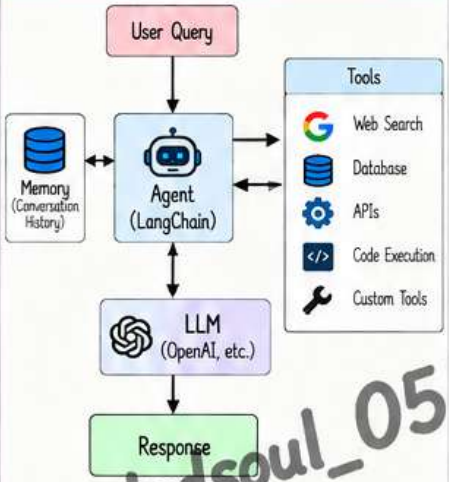
# Load and split documents
loader = PyPDFLoader("docs.pdf")
docs = loader.load()
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=200)
chunks = splitter.split_documents(docs)

# Store in vector DB (Pinecone example)
embeddings = OpenAIEmbeddings(api_key="YOUR_API_KEY")
vectorstore = Pinecone.from_documents(
    chunks, embeddings, index_name="your-index")

# Retrieval QA chain
llm = ChatOpenAI(model="gpt-4o", api_key="YOUR_API_KEY")
qa_chain = RetrievalQA.from_chain_type(
    llm=llm, retriever=vectorstore.as_retriever())

query = "What is mentioned about system design?"
result = qa_chain.run(query)
print(result)
```

AI Agent Architecture



Popular Tools & Integrations

OpenAI · Hugging Face · Pinecone · Weaviate · Chroma · Qdrant · Tavily (Web Search) · SerpAPI · SQL Database

Types of AI Agents

1 ReAct Agent	Reason + Act using tools
2 Self Ask Agent	Breaks complex questions into sub questions
3 Planner Agent	Plans steps before execution
4 Tool Using Agent	Uses external tools (search, code, APIs)
5 Multi Agent System	Multiple agents working together
6 Autonomous Agent	Works with minimal human intervention

Real World Use Cases

Document Q&A · Research Agent · Data Analysis Agent · Code Assistant · Customer Support Bot · Enterprise Knowledge Base · Automated Report Generation · Financial Analysis · Healthcare Assistant · Personal AI Assistant

Best Practices

- Use clear system prompts and tool descriptions
- Choose the right chunk size and embedding model
- Keep agent flow simple and modular
- Handle errors and fallbacks properly
- Add memory for better context
- Use guardrails and validation
- Monitor cost and token usage
- Use LangGraph for complex workflows
- Test with real data and edge cases

Common Challenges

- Incorrect tool selection
- Hallucinations in agent responses
- Poor chunking leading to bad retrieval
- High API and storage costs
- Latency in multi step workflows
- Handling long context and memory
- Debugging complex agent flows
- Security and prompt injection risks
- Maintaining accurate and up to date data

@Codedsoul_05

13. Full Stack AI Application Architecture

@Codedsoul_05

What is a Full Stack AI Application ?

- A full stack AI application combines frontend, backend, databases, AI models (LLMs), and additional services to deliver a complete real world product.
- It includes everything from user interface to AI inference, data storage, authentication, and deployment.
- Examples: AI Chatbot, Document Q&A, AI Image Generator, AI Coding Assistant, AI SaaS products, etc.

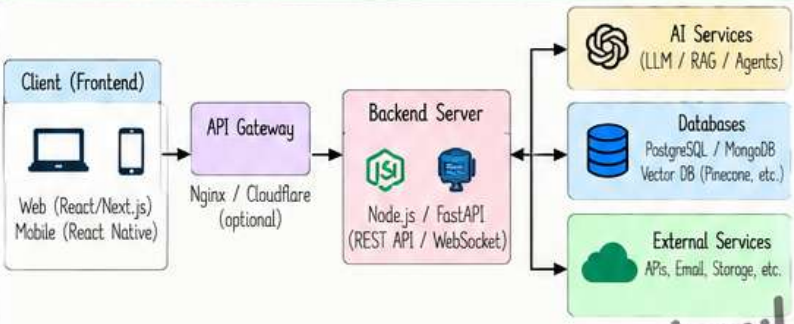
Key Components

1. Frontend (UI / UX)
2. Backend (API & Business Logic)
3. AI Layer (LLM, RAG, Agents)
4. Database (SQL / NoSQL / Vector DB)
5. Authentication & Authorization
6. Cloud & Deployment
7. Monitoring & Analytics

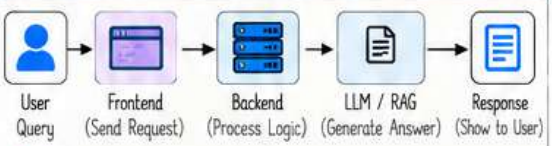
Tech Stack Overview

Frontend	React / Next.js, Tailwind CSS
Backend	Node.js / Express.js, Python / FastAPI
AI / LLM	OpenAI, Anthropic, Hugging Face, LangChain
Database	PostgreSQL, MongoDB, Pinecone / Weaviate / Qdrant
Authentication	JWT, OAuth, Clerk, NextAuth
Deployment	Vercel, Docker, AWS
Monitoring	LangSmith, Sentry, Prometheus, Grafana

High Level Architecture



Data Flow in an AI Application



1. User sends query from frontend.
2. Backend handles authentication and business logic.
3. Retrieves relevant context (if using RAG) from vector database.
4. Calls LLM / Agent to generate response.
5. Sends final response back to the frontend.

@Codedsoul_05

Frontend Example (Next.js)

```jsx
import { useState } from 'react'

export default function ChatPage() {
  const [message, setMessage] = useState('')
  const [response, setResponse] = useState('')

  const sendMessage = async () => {
    const res = await fetch('/api/chat', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ message })
    })
    const data = await res.json()
    setResponse(data.answer)
  }

  return (
    <div>
      <input value={message} onChange={(e)=>setMessage(e.target.value)}
        placeholder="Ask anything..." />
      <button onClick={sendMessage}>Send</button>
      <p>{response}</p>
    </div>
  )
}
```

Backend Example (FastAPI)

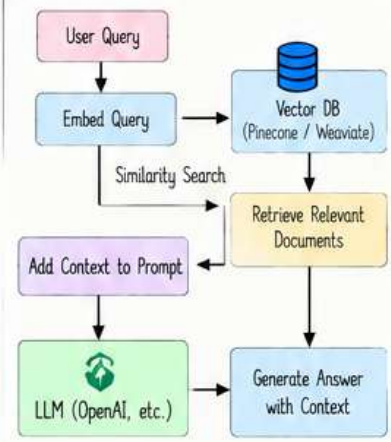
```python
from fastapi import FastAPI
from pydantic import BaseModel
from openai import OpenAI

app = FastAPI()
client = OpenAI(api_key="YOUR_API_KEY")

class ChatRequest(BaseModel):
    message: str

@app.post("/api/chat")
async def chat(req: ChatRequest):
    response = client.chat.completions.create(
      model="gpt-4o",
      messages=[{"role":"user","content":req.message}]
    )
    return {"answer": response.choices[0].message.content}
```

RAG Pipeline Example



Database Design (Example)

PostgreSQL (Relational)

users	chat_history
id (PK)	id (PK)
name	user_id (FK)
email	message
password_hash	response
created_at	created_at

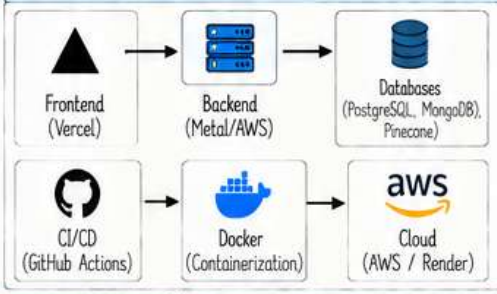
MongoDB (NoSQL)

```
{
  _id: ObjectId(),
  userId: "123",
  message: "What is AI?",
  response: "AI is ...",
  createdAt: ISODate()
}
```

Vector Database (Pinecone)

```
id: "doc1"
values: [0.12, -0.45, ...]
metadata: {
  "text": "Machine learning...",
  "source": "blog",
  "url": "https://..."
}
```

Deployment Architecture



Real World Use Cases

- AI Chatbot (SaaS)
- Document Q&A (PDF, Docs)
- AI Study Assistant (Education)
- Customer Support Automation
- AI Content Generator (Blogs, Social Media)
- AI Code Assistant (Developer Tools)
- AI Search Engine (Enterprise)
- Personal AI Assistant (Productivity)

Best Practices

- Use modular architecture (separate frontend, backend, AI).
- Implement authentication and rate limiting.
- Use vector database for knowledge based applications.
- Handle errors and fallback responses.
- Monitor API usage and costs.
- Use environment variables for sensitive keys.
- Implement caching for better performance.
- Follow security best practices (input validation, access control).
- Use CI/CD for automated deployment.
- Choose the right cloud platform based on scale and cost.

Common Challenges

- High API costs.
- Latency in LLM responses.
- Managing large documents.
- Keeping vector data updated.
- Security and prompt injection risks.
- Handling multiple users and scale.
- Integrating multiple tools and APIs.
- Debugging complex workflows.
- Maintaining context in long conversations.
- Ensuring data privacy and compliance.

14. Docker + Cloud Deployment + Production AI

@Codedsoul_05

What is Docker ?

- Docker is a containerization platform that packages your application and its dependencies into a container.
- It ensures your application runs consistently across different environments (development, staging, production).
- Useful for AI applications to package models, libraries and services easily.

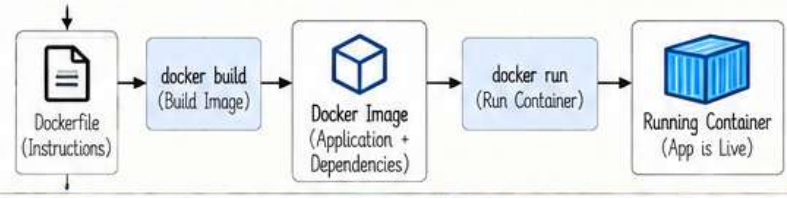
Why Use Docker for AI Apps ?

- Consistent environment across machines
- Easy dependency management (e.g. Python, ML libraries)
- Lightweight and portable
- Simplifies deployment
- Works well with cloud platforms
- Helps in scaling AI services

Cloud Platforms for Deployment

- Amazon Web Services (AWS)
- Google Cloud (GCP)
- Microsoft Azure
- Vercel (Frontend / Full Stack)
- Render (Backend)
- Railway (Full Stack)

How Docker Works ?



Dockerfile Example (FastAPI AI App)

```
# Use official Python image
FROM python:3.11-slim

# Set working directory
WORKDIR /app

# Copy requirements and install
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY . .

# Expose port
EXPOSE 8000

# Run the application
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Build and Run Docker Container

```
# Build the image
docker build -t ai-app .

# Run the container
docker run -d -p 8000:8000 --name ai-app ai-app

# Check running containers
docker ps

# Stop container
docker stop ai-app

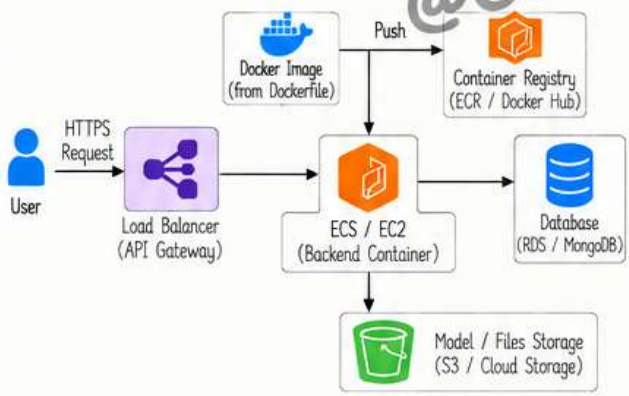
# Remove container
docker rm ai-app
```

Docker Compose (Multiple Services)

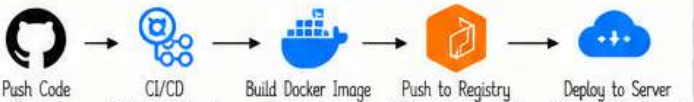
```
version: '3.8'
services:
  app:
    build: .
    ports:
      - "8000:8000"
    depends_on:
      - db
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: ai_db
    ports:
      - "5432:5432"
```

@Codedsoul_05

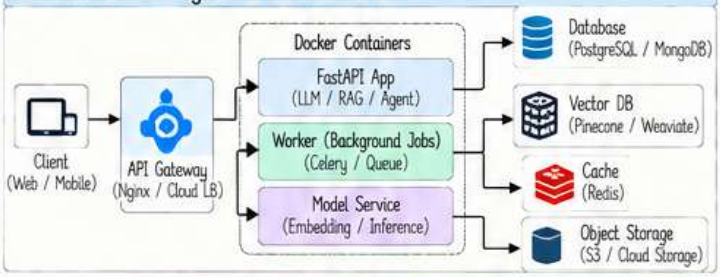
Cloud Deployment Architecture



CI / CD Pipeline (Automated Deployment)



Production AI System Architecture



Example: Deploy FastAPI AI App on AWS (EC2)

1. Launch an EC2 instance (Ubuntu)
2. Install Docker and Docker Compose
3. Clone your project code
4. Build and run the Docker container
5. Configure security group (open port 8000)
6. Use a domain name (optional) with Nginx as reverse proxy

Production Checklist

- Use environment variables for API keys
- Use a reverse proxy (Nginx)
- Enable HTTPS (SSL Certificate)
- Use monitoring (Prometheus / Grafana)
- Set up logging (CloudWatch / ELK)
- Use auto scaling (ECS / GKE / AKS)
- Implement backup for database
- Use separate environments (dev, staging, prod)
- Handle errors and fallback mechanisms
- Monitor cost and resource usage

Monitoring & Logging

- Prometheus (Metrics)
- Grafana (Dashboard)
- ELK Stack (Logs)
- CloudWatch (AWS)
- Sentry (Error Tracking)
- Uptime Robot (Uptime)

Real World Use Cases

- AI Chatbot (SaaS)
- Document Q&A (PDF / Docs)
- AI Image / Video Generation
- RAG Search (Knowledge Base)
- AI Agents (Automation)
- ML Model Deployment
- Enterprise AI Applications
- Scalable AI APIs (Production)

Common Challenges

- Large Docker image size
- Managing sensitive keys and environment variables
- Handling high traffic and scaling
- Model inference latency
- Cost of cloud deployment
- Keeping services up to date
- Debugging container issues
- Monitoring model performance in production

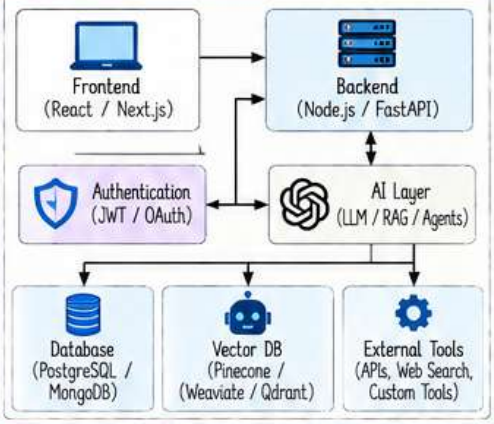
15. Full Stack AI Cheat Sheet + 100 Interview Questions

@Codedsoul_05

Tech Stack at a Glance

Frontend	React / Next.js / Tailwind CSS
Backend	Node.js /Express / Python (FastAPI)
Database	PostgreSQL / MongoDB / Vector DB
AI/ML	/ LangChain / LangGraph
Deployment	aws / AWS / Vercel
Monitoring	

Full Stack AI Architecture



Common Use Cases

- AI Chatbot (Customer Support)
- Document Q&A (PDF, Docs, Websites)
- AI Study Assistant
- AI Image / Video Generation
- RAG with Private Data
- AI Agents for Automation
- Enterprise AI Applications
- Code Assistant / Copilot
- Data Analysis with AI
- AI SaaS Products

Important Commands

Create React App	`npx create-next-app@latest`
Create FastAPI App	`pip install fastapi uvicorn main:app --reload`
Run Node.js Server	`npm init -y` `npm install express`
Run Docker	`docker build -t app .` `docker run -p 8000:8000 app`
Docker Compose	`docker-compose up -d`
Deploy on Vercel	`vercel --prod`
Deploy on AWS (EC2)	`ssh -i key.pem ubuntu@ip`
Connect PostgreSQL	`psql -U username -d dbname`
Donnect MongoDB	`mongosh mongodb+srv://...`

Environment Variables (.env)

```
# General
NODE_ENV=production
PORT=8000

# Database
DATABASE_URL=postgresql://user:pass@host:5432/db
MONGODB_URI=mongodb+srv://user:pass@cluster/db

# OpenAI
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxx

# Vector DB
PINECONE_API_KEY=xxxxxxxxx

# JWT
JWT_SECRET=your_secret_key

# Cloud / Other
AWS_ACCESS_KEY_ID=xxxxxxxx
AWS_SECRET_ACCESS_KEY=xxxxxxxxx
```

Project Structure (Full Stack AI)

```
ai-fullstack-app/
├── frontend/ (Next.js)
│   ├── app/
│   ├── components/
│   └── .env
├── backend/ (FastAPI / Node.js)
│   ├── routes/
│   ├── services/
│   ├── models/
│   └── .env
├── database/
│   └── migrations/
├── docker/
│   └── docker-compose.yml
├── scripts/
└── README.md
```

@Codedsoul_05

Key Concepts Cheat Sheet

1. **LLM**: Large Language Model for text generation
2. **RAG**: Retrieve relevant data + generate answer
3. **Embeddings**: Convert text to vectors
4. **Vector DB**: Store and search embeddings
5. **LangChain**: Build LLM applications with tools
6. **LangGraph**: Build agent workflows
7. **Agents**: Autonomous systems using LLM + tools
8. **Prompt Engineering**: Better inputs for better outputs
9. **Fine-tuning**: Train model on custom data
10. **Docker**: Containerize your application
11. **CI/CD**: Automate testing and deployment
12. **Monitoring**: Track logs, errors and performance

API Endpoints Example (FastAPI)

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class ChatRequest(BaseModel):
    message: str

@app.post("/api/chat")
async def chat(req: ChatRequest):
    response = client.chat.completions.create(
      model="gpt-4o",
      messages=[{"role": "user", "content": req.message}]
)
return {"answer": response.choices[0].message.content}
```

RAG Pipeline Cheat Sheet

1. Load Documents (PDF, Text, Web, etc.)
2. Split into Chunks (500 - 1000 tokens)
3. Create Embeddings (OpenAI / BAAI)
4. Store in Vector DB (Pinecone / Qdrant)
5. Retrieve Relevant Chunks (Similarity Search)
6. Generate Answer (LLM with Context)

Deployment Checklist

- [x] Use environment variables
- [x] Set up production database
- [x] Use Docker for consistent environment
- [x] Configure domain and SSL (HTTPS)
- [x] Use reverse proxy (Nginx)
- [x] Set up monitoring (Prometheus / Grafana)
- [x] Enable logging (ELK / CloudWatch)
- [x] Auto-deploy with CI/CD (GitHub Actions)
- [x] Handle errors and rate limits
- [x] Backup data and models
- [x] Monitor usage and costs

Best Practices

- [x] Use modular code structure
- [x] Keep API keys secure
- [x] Validate user inputs
- [x] Use proper authentication and authorization
- [x] Use caching for repeated queries
- [x] Handle errors gracefully
- [x] Keep prompts concise and clear
- [x] Use vector DB for large documents
- [x] Monitor token usage and costs
- [x] Test with real world data
- [x] Follow cloud security best practices

Common Interview Topics (100 Questions)

1	Python & Programming Basics	(10 Qs)
2	Data Structures & Algorithms	(10 Qs)
3	Database (SQL, PostgreSQL, MongoDB)	(10 Qs)
4	Web Development (React, Next.js, Node.js)	(10 Qs)
5	APIs & System Design	(10 Qs)
6	AI / ML Fundamentals	(10 Qs)
7	LLMs, RAG, LangChain, LangGraph	(15 Qs)
8	Deployment, DevOps, Cloud	(10 Qs)
9	Security, Authentication, Best Practices	(10 Qs)
10	Real World Scenarios & Case Studies	(5 Qs)

Sample Interview Questions (Selected Examples)

Python & Programming

1. What is a generator in Python?
2. Difference between list and tuple?
3. What is GIL in Python?
4. Explain decorators with an example.
5. How does async await work?

Database (SQL, MongoDB)

1. What is normalization?
2. Difference between SQL and NoSQL?
3. Write a SQL query to find duplicate records.
4. How does indexing work in PostgreSQL?
5. Explain aggregation pipeline in MongoDB.

RAG & LLMs

1. What are embeddings and how do they work?
2. How does RAG improve LLM responses?
3. Explain LangChain and its components.
4. What is LangGraph and why is it used?
5. How do you handle large documents in RAG?

Deployment & Cloud

1. How does Docker work?
2. Explain CI/CD pipeline for an AI app.
3. How will you deploy a FastAPI app on AWS?
4. What is a reverse proxy and why use Nginx?
5. How do you monitor an AI application?